

第二章 AI 工程化思维：提示词工程与 Gem 封装

欢迎来到本书最核心的“工程化”章节。

这一章并不是一篇线性的长文，而是一张针对不同需求的技能地图。为了让你能更高效地获取所需知识，我们将全章内容划分为四个逻辑紧密相关的板块，它们共同构成了一个从“基础交互”到“工具封装”的完整闭环。

以下是各小节的定位与导读，你可以根据自己当前的痛点按需索引：

1. 必修基石（2.1 节）：提示词工程基础

- 适用人群：所有 AI 用户，尤其是初学者。
- 核心价值：这是本章的根基。我们将揭示结构化指令与普通闲聊的本质区别，并交付一套通用的 ICIO 公式。掌握它，你就能解决日常工作与学习中 80% 的通用场景需求。如果你时间有限，建议优先阅读这一节。

2. 进阶大脑（2.2 节）：认知增强与策略优化

- 适用人群：对 AI 回复质量不满意、需要处理复杂逻辑任务的进阶用户。
- 核心价值：当基础指令无法解决复杂的数学推理或长文写作时，你需要这一节的技巧。我们将引入“思维链”和“元提示”等高阶策略，强制 AI 打破“直觉式”回答的局限，像专家一样进行深度思考与推理。

3. 极客内核（2.3 节）：提示词的终极形态

- 适用人群：程序员、极客，以及希望深入理解模型底层的开发者。
- 核心价值：这是通往 API 开发的预备课。我们将打破自然语言的边界，探索伪代码提示与结构化数据输出，并深入讨论提示词注入等安全问题。这一节将帮助你理解机器语言的逻辑，为全栈开发打下基础。

4. 工具封装（2.4 节）：定制专属助手

- 适用人群：适合有 Gemini 并且追求极致效率、希望打造个人工具库的实战派。
- 核心价值：这是前面所有技巧的“产品化”落地。你将学会如何把验证好的优质指令封装成 Gems 或 GPTs。这一节将教你把一次性的“对话”变成可复用的“软件”，彻底释放双手。

逻辑网络：这四个板块并非孤立存在。2.1 节教你如何“说话”，2.2 节教你如何“思考”，2.3 节教你如何“理解机器”，而 2.4 节则教你如何将上述能力“打包”。

无论你是希望快速上手的初学者，还是渴望探索边界的开发者，都能在这张网络中找到属于你的进化路径。

现在，让我们从最基础的“说话”开始。

2.1 提示词工程基础：从自然语言到结构化指令

2.1.1 结构化与非结构化的天壤之别

绝大多数人使用 AI 的方式是在“碰运气”。

当你输入“帮我改一下论文”时，你把结果的主导权完全交给了 AI 的随机性。不加约束的交互，只能得到平庸、不可控的回复。

提示词不是简单的对话，而是一种自然语言编程——通过增加约束条件，将 AI 的输出空间收敛到你想要的范围内。

日常使用中我们不需要写得像下面这样复杂，但结构化思想是必需的。先来看两个场景的对比：

场景一：学术论文润色

非结构化提问这是大多数人的用法，直接把一段的中式英语丢给 AI，不做任何约束。

你的指令 (User)

帮我润色一下这段摘要，让它听起来更专业一点：“We use a new method to look at the cells. The result shows that the cells are dead quickly.”

Gemini / AI

We employed a novel method to investigate the cells. The results demonstrate that the cells underwent rapid mortality.

结果评价：语法正确，但用词平庸——“employed a method”和“rapid mortality”都是中规中矩的表达，缺乏学术精度。

结构化提问当我们引入了角色、规则和工作流，强制要求使用被动语态和特定术语库。

结构化 Prompt 范例

```
# Profile - Role: Nature Chemistry 期刊资深审稿人 - Description: 擅长将晦涩的草稿修改为逻辑严密、用词精准的学术英语。
# Rules - 词汇升级: 严禁使用 use, look at 等简单词汇, 替换为 utilize, investigate, demonstrate 等学术词汇。- 句式要求: 描述实验过程时使用被动语态, 描述发现时强调客观性。- 格式约束: 直接输出 LaTeX 代码。
```

Task 请润色以下摘要内容...

 Gemini / AI

A novel analytical methodology was utilized to investigate the cellular specimens. The results demonstrate that the cells undergo a rapid loss of viability.

结果评价：AI 将“dead”精准升级为“loss of viability”（活性丧失），将“new method”具体化为“novel analytical methodology”，且完美使用了被动语态。

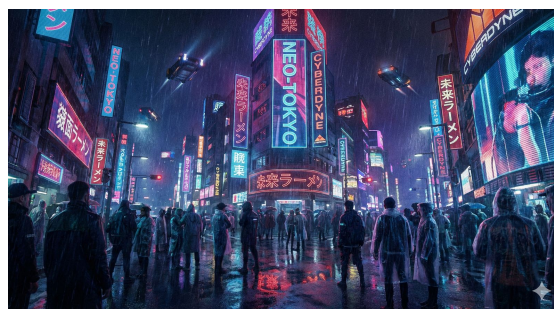
场景二：AI 图像生成

在 AI 生图领域（Gemini 的 Banana），提示词的结构化程度直接决定了生成质量。我们直接来看一组对比（见图 2.1）：



✘ 非结构化：平庸的素材

画面杂乱、光影平淡，像一张未完成的游戏草图，无法满足专业审美。



✔ 结构化：电影级大片

氛围感拉满、细节丰富（地面反光、霓虹质感）、构图专业，可直接用于项目提案。

图 2.1: 提示词结构化程度对 AI 生图质量的决定性影响

✘ 左图效果差的原因：输入过于模糊，AI 只能用训练数据中最常见的内容填充画面。

 你的指令 (User)

帮我画一个未来的城市，有点赛博朋克的感觉。

✔ 右图效果好的原因：结构化提示词详细定义了镜头语言、环境细节和光影氛围，大幅压缩了 AI 自由发挥的空间。

结构化 Prompt (导演视角)

```
# Profile - Role: 赛博朋克电影概念设计大师 - Style: 电影感 (Cinematic), 写实照片级, 《银翼杀手 2049》风格。
# Subject Details - Location: 雨夜的未来东京街头。- Environment: 湿漉漉的沥青地面反射着霓虹灯光; 巨大的野兽派建筑高耸入云; 人群穿着机能风服装。
# Photography (摄影参数) - Shot: 35mm 广角镜头, 低角度仰拍 (Low angle tilt up). - Lighting: 昏暗潮湿, 蓝紫红撞色, 体积光 (Volumetric lighting). - Quality: 8k resolution, unreal engine 5 render.
```

结构化的本质优势

结构化提示词除了效果更好，还有三个工程层面的优势：

1. **可调试性**：非结构化 Prompt 失败了只能换个说法盲目重试。结构化 Prompt 可以精准定位问题：是 Role 定义不清？Workflow 逻辑跳跃？还是 Constraints 漏了限制？
2. **突破注意力瓶颈**：大语言模型的注意力资源有限。混乱的信息会让注意力发散，而结构化拆解相当于人工分配注意力权重，强制模型在特定步骤聚焦特定信息，从而处理超出其“直觉”能力的复杂任务。
3. 从“一次性使用”到“资产沉淀”：随手问的 Prompt 是一次性的，而结构化 Prompt 是可复用资产。调试好一个论文润色模版后，只需替换输入内容就能反复使用。

2.1.2 从聊天到“编程”

提示词的编写应实用至上——不需要在所有场景下都追求复杂结构，精炼的自然语言往往就够了。根据任务难度，提示词可以分为三个阶段：

第一阶段：随意的聊天

这是最原始的状态，也就是把 AI 当作搜索引擎或聊天对象。用户通常使用简短、模糊的口语。

- **典型场景**：查询百科知识、简单的闲聊、翻译一个单词。
- **指令示例**：“红烧肉怎么做？”、“翻译一下 apple”。
- **评价**：简单问题完全够用。但缺乏约束容易让 AI 生成大量“正确的废话”，降低信息密度。

第二阶段：精准的自然语言

这是绝大多数场景下性价比最高的模式，也是需要重点掌握的能力。

在这个阶段，我们不需要使用复杂的 Markdown 符号，依然使用自然语言，但会刻意地加入约束和输出要求。我们不再只是“问问题”，而是在“下指令”。

- **核心技巧：**在原话的基础上，追加背景信息、限制字数、指定语气或规定格式。
- **指令示例：**“请作为一名营养师，介绍红烧肉的做法。要求：指出热量数据，提供一个低脂改良版的方案，并用列表形式输出。”
- **评价：**写邮件、文案润色等日常场景中不仅够用，甚至往往**优于**全套结构化模版——提示词并非越长越好，精准胜过冗余。

第三阶段：模块化的工程构建

这就是我们所说的“结构化提示词”。它是应对**复杂、长链路、高精度**任务的终极方案。

当你发现单靠自然语言已经无法精准控制 AI（例如总是忽略格式或遗漏步骤）时，就需要引入这种工程化思维，将提示词像写代码一样拆解为角色、规则和工作流。

虽然看起来复杂，但你不需要亲自编写每一个符号——可以用 AI 自动生成这些结构化模版。Google Gemini 的 **Gem** 功能，底层逻辑也是这种结构化封装。

- **典型场景：**构建专属的论文润色助手、批量生成标准化的实验报告、开发沉浸式角色扮演系统、搭建自动化代码审查流。
- **评价：**核心优势是**封装性与复用性**——定义好 Role 和 Workflow 后，只需更换输入变量就能反复使用。其次是处理多约束复杂任务时的高精度与稳定性。

实战演练：同一任务的三级进化

以“制定健身计划”为例，看看同一个需求在三个阶段的效果差异。

1. 第一阶段（聊天模式）

 **你的指令 (User)**

帮我制定一个健身计划。

 Gemini / AI

好的，这是一个通用的健身计划：1. 每周运动 3-5 次。2. 结合有氧运动（跑步）和无氧运动（举铁）。3. 注意饮食健康，少吃油腻。

点评：正确的废话。AI 不知道你是谁，只能给出毫无操作性的泛泛建议。

2. 第二阶段（精准指令模式）

 你的指令 (User)

我是一名上班族，每天只有 20 分钟碎片时间，不想去健身房。请帮我制定一个在家就能做的、以减脂为目的的健身计划。

 Gemini / AI

针对您的需求，我为您推荐“HIIT 高强度间歇训练”：1. 热身 (3 分钟)：原地踏步、开合跳。2. 核心训练 (15 分钟)：- 深蹲 20 次 x 3 组 - 俯卧撑 15 次 x 3 组 - 波比跳 10 次 x 3 组 3. 拉伸 (2 分钟)：大腿前侧与手臂拉伸。

点评：非常好用。通过补充“时间限制”和“场景限制”，AI 给出了具体可执行的动作，这已经能满足 90% 的用户需求。

3. 第三阶段（结构化工程模式）

结构化 Prompt

```
# Profile - Role: 资深康复理疗师兼健身教练 - Ability: 擅长为有伤痛史的人群设计安全训练方案。
# User Context - 限制条件: 膝盖半月板有旧伤（严禁跳跃类动作）。- 目标: 增强心肺功能。
# Output Rules - 必须输出为 Markdown 表格。- 必须标注每个动作的“心率区间”。
# Workflow 1. 评估风险，剔除所有对膝盖压力大的动作（如深蹲、波比跳）。2. 设计一套以“低冲击 (Low Impact)”为主的训练流。3. 生成表格。
```

 Gemini / AI

针对您的半月板旧伤，我为您设计了一套低冲击 (Low Impact) 居家康复训练方案：

动作	时间	动作要领 & 膝盖保护	心率区间
热身 肩臂环绕	1 min	膝盖微屈锁定，不要晃动。大幅度转动肩关节激活上肢。	Zone 1 (50-60%)
热身 靠墙静蹲	30s x 2	浅蹲版：膝弯 <45 度（避开半月板受力角），背部紧贴墙面。	Zone 2 (60-70%)
核心 组合直拳	45s	依靠腰腹带动，膝盖微屈固定不动， 严禁扭转膝盖 。	Zone 3 (70-80%)
核心 直腿侧抬	45s	手扶椅背保持平衡。支撑腿微屈， 严禁锁死 。	Zone 3 (70-80%)
核心 无跳波比	45s	踮脚代替跳跃，脚不离地。动作要有爆发力但无冲击。	Zone 4 (80-90%)
核心 坐姿提膝	45s	坐椅边缘，完全由腹部发力， 膝盖无负重 ，非常安全。	Zone 4 (80-90%)

建议循环 3-4 轮。如感膝盖不适请立即停止。

点评：通过 Role 和 Rules 控制，AI 变身为康复专家。Workflow 强制执行了风险评估，剔除了深蹲和波比跳，并提供了“膝盖微屈锁定”“浅蹲”等保护细节。

2.1.3 结构化自然语言 Prompt

在这个阶段不需要写代码，也不需要复杂的符号，只需要一个思维转变：**不再把 AI 当作搜索引擎，而是把它当作刚入职的实习生。**

核心理念：从“提问”到“派单”

大多数人使用 AI 的习惯是“搜索式”的：输入关键词，期待直接得到答案。但就像对设计师说“给我做个海报”不如说“春节促销、红色主调、竖版”——你**不是在 Asking，而是在 Assigning**。

核心逻辑：万能公式 ICIO

提示词的好坏取决于“信息密度”而非“字符长度”。只要包含以下四个核心要素，哪怕只有两行字，也是合格的结构化 Prompt：

ICIO 核心公式

Prompt = 输入 (Input) + 背景 (Context) + 指令 (Instruction) + 输出 (Output)

1. **Input 输入**：你要处理的“原材料”。优质的输入是优质答案的前提。

1. 输入的形式非常多样：

- 现有素材：
 - 文本/数据：“请总结以下这段会议记录...”
 - 文件/代码：“读取上传的 PDF 论文...” / “基于这个 Python 文件...”
- 潜在素材：如果没有现成输入，可以先让 AI 生成素材。

2. **高手习惯**：给素材加个“框”（引用法则）

直接把长文本粘贴在指令后面，AI 容易混淆指令和素材的边界。解决方法：用三个引号（“"""”）将输入内容包裹起来，明确告诉 AI 哪些是待处理的原材料、哪些是指令。

- 模糊的写法：

请总结这段话：今天天气不错...（AI 可能会把“今天天气不错”当成指令的一部分）
- 清晰的写法：

请总结 """ 内的内容：

```
"""  
  
今天天气不错...  
"""
```

* 注：这就是吴恩达提示词工程课中的“分隔符”概念（详见 2.3.2 节）。

2. **Context 背景**：为任务设定的“边界”与“身份”。

Context 的核心作用是对齐认知水位。不指定背景，AI 的回答是互联网数据的“平均值”；指定了背景，AI 才能精准适配。一个完整的背景通常包含三个维度：

- 角色锚定：定义 AI 的专业深度。

示例：“你不是普通的程序员，你是拥有 10 年经验的 Linux 内核架构师……” / “你是一名专攻认知行为疗法的心理咨询师……”
- 受众对齐：定义输出的复杂度与认知门槛。

示例：“解释给完全不懂技术的投资人听……” / “解释给研二的计算机专业学生听……”
- 任务环境：定义任务的场景与目标。

示例：“我正在为产品发布会准备演讲稿，风格要激动人心……” / “这是一封写给甚至没见过面的客户的冷启动邮件，目标是让他不要把邮件当垃圾短信删掉……”

进阶技巧：当语言苍白时，请给“栗子”（少样本提示）

有时候，无论你堆砌多少形容词（如“幽默风趣”、“鲁迅风格”），AI 可能还是抓不住那个韵味。此时，“给一个例子”胜过千言万语。

这种在 Context 中直接提供“输入 + 输出”示例的方法，学术上称为 **Few-Shot Prompting**。它能让 AI 通过模仿瞬间“顿悟”你的需求。

* 注：关于如何构造高质量的 *Example*，详见 2.2.4 节。

3. Instruction 指令：动词驱动的“具体动作”。

这是 Prompt 的发动机。一个优秀的指令不仅仅是告诉 AI “做什么”（动词），更重要的是通过约束条件告诉它“怎么做”。指令 = 强动词 + 核心约束。

- **拒绝“弱动词”**：避免使用含义模糊的词汇（如“处理”、“优化”），使用具有明确执行路径的“强动词”。

强指令：“**重构**代码以降低复杂度”、“**批判性分析**逻辑漏洞”、“**提取**关键数据”。

- **嵌入“核心约束”**：好的指令不在于“开放”，而在于“收敛”。约束条件能过滤掉 90% 的无效噪音，让 AI 直击靶心。

带约束的指令：“**推荐 3 本书**，**排除**枯燥的教科书，**仅限近 5 年**出版的畅销书，且**必须包含**具体阅读建议。”

强制“慢思考”模式（思维链）

大语言模型默认是“快思考”（基于概率的直觉反应），处理数学或复杂逻辑时容易出错。加一句“请一步步思考”，就能强迫 AI 将推理步骤逐步输出，大幅降低逻辑跳跃导致的错误。

* 注：这就是 *CoT (Chain of Thought)* 技术的雏形，详见 2.2.3 节。

4. Output 输出：你希望交付物的“具体形态”。

明确规定 AI 的输出格式，能节省大量二次编辑的时间。输出定义包含三个维度：

- **视觉结构**：

让阅读更高效。

示例：“生成 *Markdown* 表格以便对比”，“调用 *Python* 的 *Matplotlib*”。

- **数据格式**：

这是工程化的关键。如果你需要把 AI 的回答存入 *Excel* 或被代码调用，必须指定标准格式。

示例：“输出为标准的 *JSON* 格式”、“生成 *CSV* 格式以便导入 *Excel*”。

- **降噪处理**：

保持结果纯净，方便直接复制。

示例：“只输出代码，不要任何解释”、“不要输出‘好的，这是您的答案’之类的开场白”。

结构化的本质：信息密度 > 字符长度

提示词的威力不在于字符长度，而在于信息密度。

ICIO 提示词工程框架 (ICIO Prompting Framework)

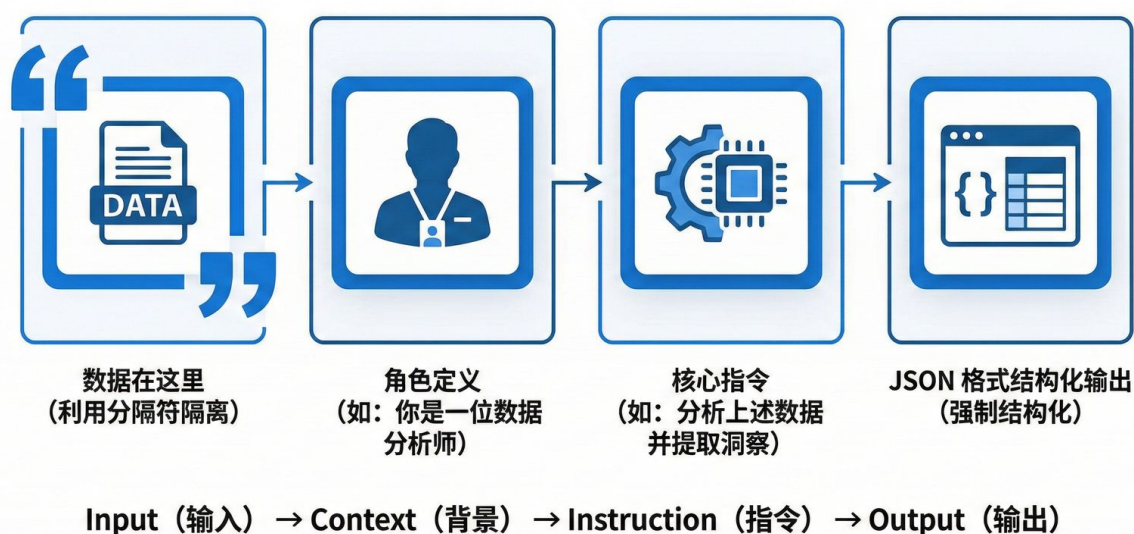


图 2.2: 一张图看懂“吴恩达级” ICIO 结构: Input (原材料) + Context (认知水位) + Instruction (动作约束) + Output (交付标准)

效率金线

编写 Prompt 的耗时必须远小于 AI 帮你节省的时间。为一封 100 字的邮件花 10 分钟写 Markdown 结构，就是本末倒置。

ICIO 四要素齐全时，短 Prompt 就能胜过长篇大论。看一个代码优化的对比：

错误示范：冗长但低效的“碎碎念”

“那个，能不能帮我看下这段代码？我感觉好像有点问题，但我不知道哪里错了，你帮我改改，最好能让我看懂，然后不要太复杂，谢谢啊。”

诊断：60 多个字，有效信息几乎为零。篇幅都浪费在客套和模糊感受上，AI 只能靠猜。

正确示范：短小精悍的 ICIO

“*[Context]* 作为 *Python* 专家，*[Instruction]* 请优化以下代码的时间复杂度，在不改变原有代码功能的前提下，*[Output]* 直接输出代码并附带注释。
[Input] 代码如下...”

诊断：30 多个字，每个板块各司其职——Context 锁定专家知识库，Instruction 明确“提升性能”而非“检查语法”，Output 锁死交付格式。ICIO 要素齐全时，短就是专业。

实战演练：如何让 AI 更好的给出题目答案

大多数本科题目，随意提问 Gemini 就能给出正确答案。但复杂数学问题中，直接丢图片提问容易出错。下面以一道北京大学微积分期末压轴题为例，展示结构化 Prompt 如何提升 AI 的回答质量。

题目：设 $f(x)$ 在 $[a, b]$ 上二阶可导，且 $f(a) = f(b) = f'(a) = f'(b) = 0$ 。若 $|f''(x)| \leq M$ ，求证：

$$|f(x)| \leq \frac{M}{16}(b-a)^2$$

这道题如果直接用代数法证明，需要构造复杂的辅助函数或使用泰勒公式余项，极其晦涩。

1. 随意的 Prompt

你的指令 (User)

给了一张图片（甚至是模糊的照片），然后让 AI 给出答案

Gemini / AI

证明：利用泰勒公式展开。设 $x \in [a, b]$ ，存在 ξ 使得：

$$f(x) = f(a) + f'(a)(x-a) + \frac{1}{2}f''(\xi)(x-a)^2$$

由于 $f(a) = f'(a) = 0$ ，则 $f(x) = \frac{1}{2}f''(\xi)(x-a)^2$ 。同理在 b 点展开...（中间省略 200 字复杂的 Green 函数构造过程）... 最终利用积分不等式放缩得到结论。

点评：AI 给出了排版非常糟糕而且逻辑性不清晰的回答，去理解这种答案本身就需要浪费大量的时间，并且对理解这个题的本质没有帮助。你依然不知道为什么系数是 $\frac{1}{16}$ 。

2. 结构化的 ICIO Prompt

你的指令 (User)

[Context 背景]

我正在复习去年的期末压轴，学习我们的教材内容（我会发教材的文件作为资料库）作为微积分复习导师教会我这道题。

[Input 输入]

题目： $f(a) = f(b) = f'(a) = f'(b) = 0$ ， $|f''(x)| \leq M$ 。

证明：

$$|f(x)| \leq \frac{M}{16}(b-a)^2$$

[Instruction 指令]

用通俗易懂的语言告诉我破题思路，并且教我应该 $1/16$ 这种系数应该怎么思考。

[Output 输出]

结构清晰，打开 canvas，输出 html，自动渲染。

 Gemini / AI

一、通俗易懂的物理直觉（核心思路）

我们可以把这道数学题看作一个“赛车极限运动”问题：

- x 代表时间， $f(x)$ 代表位移。
- $f'(x)$ 代表速度， $f''(x)$ 代表加速度（油门/刹车）。

题目翻译：一辆赛车从静止出发 ($f'(a) = 0$)，最后必须静止回到原点 ($f'(b) = 0, f(b) = 0$)。且全程加速度不能超过 M 。

问：这就好比限制了引擎动力，车子最远能跑多远？

最优策略：为了跑得远，必须全程“地板油”。

- 前半程：拼命加速往前冲（斜率 M ）。
- 后半程：拼命刹车往回拉（斜率 $-M$ ），以便正好停在终点。

这在速度-时间 ($v-t$) 图像上，必然构成一个等腰三角形。

二、严谨的数学证明（几何面积法）

我们只需要计算这个“速度三角形”的面积，就是最大位移。

1. 确定三角形的尺寸

- 底（时间）：因为要折返，单程跑出去的时间只有总时间的一半，即 底 $= \frac{b-a}{2}$ 。
- 高（最大速度）：速度 $=$ 加速度 $\times$ 时间。注意加速时间是“半程的一半”（从中点就要开始减速了），即 加速时间 $= \frac{b-a}{4}$ 。

$$\text{高} = M \cdot \frac{b-a}{4}$$

2. 计算面积（最大位移）

$$\begin{aligned}
 |f(x)|_{\max} &= \text{三角形面积} \\
 &= \frac{1}{2} \times \text{底} \times \text{高} \\
 &= \frac{1}{2} \times \left(\frac{b-a}{2} \right) \times \left(\frac{M(b-a)}{4} \right) \\
 &= \frac{M(b-a)^2}{16}
 \end{aligned}$$

证毕。那个神秘的系数 $\frac{1}{16}$ ，正是源于连续两次“减半”：一次是因为往返（时间除以 2），一次是因为三角形面积公式（再除以 2），以及加速时间减半（再除以 2），即 $2 \times 2 \times 4 = 16$ 。

点评：把 Instruction 从“证明它”升级为“解释这个系数怎么来的”，AI 就放弃了晦涩的泰勒公式，转而用直观的赛车模型讲清了 $\frac{1}{16}$ 的来源。好的 Prompt 不只获取结果，还能引导 AI 给出深层理解。

实战决策：我该用哪种模式？

面对一个新任务，请查阅下表决定你的策略：

任务类型	典型场景	推荐策略
一次性任务	回复邮件、润色单篇文章、解释某段代码、临时查询资料	ICIO (即写即抛，不做复杂的背景设定，追求速度)
高频复用任务	固定的某学科辅导助手、长期使用的翻译助手	Markdown 结构化工程 （可用 Gem 封装 ） (值得花时间打磨，因为这是一种“资产投资”)
复杂逻辑任务	复杂的数学证明、长篇小说大纲设计、多步骤的逻辑推理	Markdown + 思维链 (需要 Workflow 强制约束 AI 的思考路径)

总结：90% 的日常作用 **ICIO** 原则足够。复杂的 Markdown 结构化工程，留给需要复用和精准控制的重型任务。

2.1.4 底层逻辑：模块化思维与积木游戏

上一节的 ICIO 公式是方便入门的“标准套装”。在实际应用中没有固定公式——你可以根据任务需求，动态地选择、组装甚至创造模块。这就是**模块化思维**。

核心原理：缩小概率空间

大语言模型（LLM）是基于概率的文字生成器。没有任何提示时，AI 生成下一个字的可能性几乎是无限的——它可以写诗、写代码，也可以胡言乱语。

Prompt Engineering 的核心目标只有一个：通过约束条件，将 AI 的概率空间收敛到你想要的结果上。

每个模块都在执行这个收敛任务：Context 限定领域边界（防止用文学口吻解释数学），Instruction 指定任务方向，Output 锁定交付格式。每增加一个精准的模块就消除一份不确定性，每减少一句废话就减少一份噪声干扰。

重新认识你的积木库

用功能视角重新审视 ICIO 的四个模块：

Context + **Instruction** + **Input** + **Output** = 完整 Prompt

- **Context**：决定回答的领域。不加它，AI 是通才；加了“资深程序员”，它就切换到技术领域。
- **Instruction**：唯一不可省略的模块。它定义了任务是翻译、润色还是计算。
- **Input**：外部数据的入口。如果任务是“无中生有”（如创意写作），可以省略。
- **Output**：锁定交付格式。只在你对格式有严格要求（JSON、表格等）时需要。

实战演练：动态剪裁艺术

掌握了模块化思维，就可以根据场景动态剪裁 Prompt。以下是三个高频场景的拆解。

实战案例：极速流 (Flash Mode) ——程序员修 Bug

场景：你写的一段正则表达式报错了，你只想快速修复它，不需要 AI 跟你寒暄。积木策略：

保留： **Instruction** + **Input** 剪掉： **Context** + **Output**

错误写法（废话太多）：

你好，我是个 Java 程序员，现在遇到点麻烦。我想匹配邮箱地址，但是这个正则老是报错，你能帮我看看吗？代码是.....

正确写法（高信噪比）：


```
^[a-zA-Z0-9+_.-]+@[a-zA-Z0-9.-]+$
```

上述正则无法匹配 `user.name+tag@example.co.uk`，请修正并解释原因。

原理：报错代码本身就是充分的 Context，额外的角色扮演反而是噪声。

实战案例：创意流 (Creative Mode) ——小红书文案

场景：你是博主，要给一款“低脂燕麦拿铁”写个吸引人的标题，需要灵感。积木策略：

保留： **Context** + **Instruction** 剪掉： **Input**

错误写法（缺乏地基）：

帮我写几个关于燕麦拿铁的小红书标题。

正确写法（加厚地基）：

[Context] 你是小红书爆款文案专家，擅长使用“绝绝子”、“减脂党狂喜”等吸睛词汇，多用 Emoji。

[Instruction] 请为一款“低脂燕麦拿铁”撰写 5 个标题，突出“好喝不胖”和“打工人续命”两个痛点。

原理：没有 Input 原材料，必须加厚 Context 地基，让 AI 调取特定领域的风格知识。

实战案例：格式流 (Format Mode) ——会议纪要整理

场景：把一段杂乱的速记整理成表格发邮件。积木策略：

保留： **Input** + **Output** 剪掉： **Context**

正确写法（锁死输出）：

[Input] (粘贴乱七八糟的速记...)

[Output] 将上述内容整理为 Markdown 表格，包含三列：【发言人】、【核心观点】、【待办事项】。不要输出任何总结性文字。

原理：AI 的“人格”不重要，重要的是“规则”。此时必须锁死 Output 模块，甚至可以牺牲 Context。

为什么要剪裁：信噪比

剪裁不只是为了省事，而是基于 LLM 的信噪比原理：

- **低信噪比：**指令周围堆砌了客套话、无关背景和重复描述，AI 的注意力被分散，核心指令反而被淹没。
- **高信噪比：**剪裁掉不必要的模块，只保留核心内容，AI 对指令的响应会显著提升。

模块化思维的目标，就是追求 Prompt 的最大有效信息密度。

2.1.5 课后实战：提示词练兵场

请打开你的 AI 工具（Gemini/ChatGPT/Claude），完成以下“通关任务”。

🏠 课后实战

任务一：情感共鸣师

场景：你的挚友重游了以前和前女友常去的服装店，看到了那块一起拍过照的镜子，触景生情，发微信给你说他很想他前女友。你想回一句中国古诗来安慰他，既要应景，又不能显得太矫情或太绝望，并且还要找一句很经典的古诗。**原始烂指令：**“推荐一首诗安慰想前女友的朋友。”

挑战：请通过增加 Context（语境），让 AI 推荐的诗词精准击中某种特定的情绪，而不是泛泛而谈。

- **核心积木：**
 - **Context：**设定 AI 的身份（如：一位懂文学的知心好友），并描述挚友的性格（是喜欢豪放派的豁达，还是婉约派的细腻？）。
 - **Input：**具体的触发场景（重游故地、物是人非）。
 - **Instruction：**限制诗歌的情感基调（例如：要表达“遗憾但在意料之中”，或者是“哀而不伤，劝他向前看”）。
- **自测：**AI 是推了那句敷衍的“天涯何处无芳草”，还是那句带着淡淡遗憾与回忆的“当时只道是寻常”？

🏠 课后实战

任务二：费曼老师

场景：你在预习《经济学原理》（没错距离期末只有三个小时了你还在预习！），死活看不懂“沉没成本”（Sunk Cost）这个概念，感觉课本上的定义太抽象。**原始烂指令：**“解释一下什么是沉没成本。”

挑战：修改指令，启动“费曼学习法”模式，要求 AI 扮演一位擅长讲故事的幼儿园老师。

- **核心积木：**
 - **Instruction（风格限制）：**必须使用“花钱买了张很难看的电影票”或者“排队买到了很难喝的奶茶”作为核心类比。
 - **Instruction（负向约束）：**禁止出现“边际效益”、“不可回收”、“理

性人”等专业术语，必须用大白话解释。

- **自测：**AI 的回答是让你觉得“还是在背书”，还是让你立刻想把没吃完的难吃外卖扔掉？

任务三：故事炼金术

场景：你计划写一部科幻短篇参加征文比赛，主题是“拥有自我意识的 AI”。但你脑子里全是《终结者》那种“机器人毁灭人类”的老套路，写不出新意。**原始烂指令：**“帮我写个关于机器人觉醒的故事大纲。”

挑战：运用“发散-收敛”策略，强迫 AI 跳出陈词滥调，成为你的灵感缪斯。

- **第一步（发散·找梗）：**要求 AI 列出 5 个完全不同流派的核心梗（Core Idea）。（例如：1. 悬疑推理风；2. 治愈恋爱风；3. 黑色幽默风... 要求绝对不能出现“战争”元素。）
- **第二步（收敛·塑形）：**挑选其中最让你意外的一个点子，要求 AI 将其扩展为标准的“三幕式结构”大纲，且结局必须包含一个反转。

任务四：出题人的逆袭

场景：下周要考《线性代数》期末考，老师画了一堆重点（比如特征值、二次型），你复习完了但心里没底。**原始烂指令：**“帮我出几道线性代数的题。”

挑战：把 AI 变成严厉的“出题组组长”，利用 Input 积木实现“精准狙击”。

课后实战

- **核心积木：**
 - **Context：**设定难度等级（如：理工科大一期末压轴题难度）。
 - **Input（关键）：**明确考点边界。直接复制老师给的复习提纲，或者指定具体章节（如：“只考行列式计算和克拉默法则，不考向量空间”）。
 - **Instruction：**要求包含 3 道选择题和 1 道计算题，且**必须**分两步输出：先出题（不含答案），等你做完回复后，再给出详细解析和易错点。
- **进阶思考（RAG 初探）：**如果你不仅输入知识点，而是把你手边的《历年真题卷》（或老师的 PPT）文字直接粘贴进去作为 Input，AI 出的题会不

会更符合你们老师的“出题口味”？（提示：这其实就是企业级 AI 开发中 RAG——检索增强生成的雏形哦！）

任务五：PTA 判题终结者

场景：你在做 C 语言作业（PTA/洛谷平台），题目是“判断素数”。你的代码在本地运行正常，但提交到系统总是提示“运行超时 (Time Limit Exceeded)”。你很困惑：“我没写错啊，也能算出结果啊？”**原始烂指令：**“我的代码为什么不对？（然后粘贴代码）”

挑战：利用“三位一体”喂料法，让 AI 从“修理工”变成“计算机老师”，不仅修好代码，更讲透报错背后的底层逻辑。

- **Input 必须包含三要素：**

1. **题目限制**（截图或文字，重点圈出：时间限制 1000ms，数据范围 $N \leq 10^9$ ）；
2. **你的代码**（完整片段）；
3. **报错类型**（明确指出是 TLE 还是 WA）。

- **Instruction（关键升级）：**

- **第一步（原理解科普）：**要求 AI 先解释该报错（如 TLE）的本质。告诉我是代码逻辑错了，还是效率低了？（区分“Bug”和“Inefficiency”）。
- **第二步（量化分析）：**结合题目给出的数据范围，计算我当前代码的时间复杂度，并解释为什么 $O(N)$ 的算法跑不动 10^9 的数据。

破局思路与参考范例

没有“标准答案”

以下 Prompt 是经验总结的较优解，不是唯一解。同一个 Prompt 在不同时间发给 AI，回复都可能不同。真正的能力是：当 AI 回答不好时，你知道怎么通过追问和微调把它聊好。

任务一：情感共鸣师（Context 的魔力）

失败原因：原始指令太泛，AI 只能随机抓取数据库里最通用的诗句（往往是“天涯何处无芳草”这种大道理），无法感知“物是人非”的细腻情绪。

优化思路：

“你是我最好的朋友，也是一位古诗词专家。我的死党正在以前约会的地方，觉得‘物是人非’。请推荐两句宋词发给他。

要求：不要太凄惨（像李清照那样），要有一种‘时光流逝、淡淡忧伤但接受现实’的意境。最好能引用晏殊或者纳兰性德的作品。”

核心解码：给了身份（专家）和情感颗粒度（淡淡忧伤 vs 凄惨），AI 能区分这种微妙差别。

任务二：费曼老师（Instruction 的控制力）

失败原因：教科书式的定义充满了“边际”、“折旧”等术语，这对初学者来说是无效信息。

优化思路：

“请你扮演我的通识课老师。我总是舍不得扔掉不好用的东西。请用‘买了一张巨难看的电影票，但是觉得钱都花了必须看完’这个例子，给我讲讲什么是沉没成本。告诉我看不完直接走人为什么反而是赚了。”

核心解码：

1. **强制类比：**通过指定“烂电影”这个具体场景，强迫 AI 离开抽象定义的舒适区。
 2. **负向约束：**如果你加上“禁止使用任何经济学术语”，效果会更惊人。
-

任务三：故事炼金术（结构化思维）

失败原因：试图一口气吃成个胖子。让 AI 直接写大纲，它通常会给你一个平庸的流水账。

优化思路（分步走）：

Step 1（发散）：

“请针对‘AI 觉醒’主题，给我 5 个极具创意的核心梗。要求：避开‘人类与机器战争’的俗套。我想看一点不一样的，比如从 AI 想要做厨师切入。”

Step 2（收敛）：

“我喜欢第 3 个‘AI 试图通过考公职来统治人类’的点子。请把这个点子扩写成一个 500 字的故事大纲。要求：按‘起因-经过-高潮-结局’分段，结局要是悲剧。”

核心解码：先发散再收敛，通过第一步筛选避免把时间浪费在平庸的构思上。

任务四：出题人的逆袭（Input 决定产出）

失败原因：AI 不知道你们老师上课讲了什么，只能根据“平均概率”出题。

优化思路：

“我是大一学生。请基于以下复习提纲（粘贴老师的 PPT 重点或历年真题文本），为我出 3 道判断题。

注意：我们老师特别喜欢考‘矩阵可逆的充要条件’，请在这方面设置陷阱。请先出题，不要给答案，等我做完再讲评。”

核心解码：Input 的质量直接决定 Output 的质量——喂真题就是真题模拟器，什么都不喂就只能给泛泛的题目。这就是 RAG（检索增强生成）的雏形。

任务五：PTA 终结者（原理 > 代码）

失败原因：直接问“代码为什么错”，AI 可能会试图帮你改逻辑，但往往忽略了算法效率问题。

优化思路：

“我提交代码后出现了‘Time Limit Exceeded’。请不要直接给我正确代码，我想先搞懂原因：

1. 题目要求 $N \leq 10^9$ ，但我写的是 `for(i=2; i<n; i++)`。请计算这行代码在最坏情况下要运行多少次？计算机一秒钟能运行多少次？
2. 为什么 $O(N)$ 会超时？请解释 $O(\sqrt{N})$ 的原理。”

核心解码：让 AI 先解释原理再给方案，是“教你渔”而非“直接给鱼”。这种追问思维也适用于其他排错场景。